

Email Transport Security in Practice: Email Client and Server Selftest Service

Sofya Generalova
Hasso Plattner Institute

University of Potsdam

Potsdam, Germany

Sofya.Generalova@hpi.uni-potsdam.de

Jhannes Ernesto Reimann

Hasso Plattner Institute

University of Potsdam

Potsdam, Germany

Jhannes.Reimann@hpi.uni-potsdam.de

Abstract—Email is a critical channel for digital communication, and Transport Layer Security (TLS) is commonly employed to secure messages in transit. For legacy reasons, TLS in email is typically not enforced by default but is instead negotiated through the STARTTLS extension. The effectiveness of this approach depends on the proper client-side implementation and server configuration. If misconfigured, such deployments may be susceptible to downgrade attacks by an active attacker on the network. In this work, we build upon existing research by designing and implementing a user-friendly web-based tool that enables users to assess the transport security properties of their email clients with respect to STARTTLS-related vulnerabilities. The tool does not require specialized technical knowledge or any environment setup. By guiding users through a structured evaluation process via a web interface, our tool lowers the barrier to practical client-side security assessment. In addition, we provide a server-side scanner that allows administrators to verify TLS configurations and detect common misconfigurations. Together, these tools aim to make email TLS evaluation more accessible to both end users and server operators.

Index Terms—email, security, STARTTLS, downgrade, TLS

I. INTRODUCTION

Email remains one of the most widely used forms of digital communication, often carrying sensitive personal and organizational information [1]. Ensuring the security of emails is therefore critical. Transport Layer Security (TLS) is widely deployed to protect email traffic in transit, providing encryption and authentication between clients and servers. However, TLS in email is often “opportunistic”, which means that it will be applied to the connection if both parties support it. This is done via the STARTTLS extension of the original email protocols. Inconsistent implementations or misconfigured servers can introduce vulnerabilities that allow an active attacker to bypass TLS protections entirely.

Historically, two approaches to TLS support have been adopted in email protocols: implicit TLS on dedicated ports, which is generally considered secure, and opportunistic TLS via the STARTTLS [2] security upgrade mechanism, which can be stripped or bypassed if the client or server does not strictly enforce TLS-only connections. This discrepancy has created a gap between the theoretical security guarantees of TLS and its real-world deployment, leaving many email users unknowingly exposed.

Motivated by these challenges, this work addresses three main problems. First, there is limited transparency regarding the behavior of email clients, as well as a lack of tools that allow end users to easily assess their email client’s security in practice. Second, email administrators require guidance and tools to verify that server-side TLS-related configurations are properly enforced. Third, it remains unclear whether known STARTTLS downgrade vulnerabilities have been addressed in current versions of popular email clients.

To address these issues, this project makes three key contributions. First, we develop a web-based self-test service that enables end users to evaluate the security of their email clients specifically with respect to STARTTLS-related vulnerabilities. The service is designed for maximum accessibility: it requires no installation, no local configuration, and no prior technical expertise, allowing anyone with a web browser to perform a meaningful assessment of their email client TLS negotiation. Second, we provide a scanning tool for email administrators, which facilitates the verification of server-side configurations and helps detect common misconfigurations that could make servers vulnerable to downgrade attacks. Third, we conduct a preliminary evaluation of a current email client against the full T1–T4 STARTTLS downgrade test matrix, comparing our findings with those of a recent large-scale study [3] and identifying previously unreported user-dependent risk factors. By focusing on STARTTLS-specific issues and targeting both client- and server-side security, these tools and findings lower the barrier for practical email transport security assessment and promote broader adoption of robust TLS practices.

II. BACKGROUND AND RELATED WORK

A. Email Protocols

The operation of email relies on a set of established protocols to facilitate both sending and receiving messages. Simple Mail Transfer Protocol (SMTP) [4] is responsible for transmitting emails, whereas Post Office Protocol (POP3) [5] and Internet Message Access Protocol (IMAP) [6] enable clients to retrieve their mail from the server. POP3 transfers emails from the server to the client and typically purges them from the server, while IMAP, a more modern protocol, maintains a synchronized state between the server and multiple clients. While there are newer email protocols available,

SMTP, IMAP, and POP3 are considered standard [7] and are widely supported.

To safeguard email transmission, TLS is deployed either implicitly or opportunistically. Implicit TLS is typically indicated by an S appended to the protocol name to signify that TLS is applied from the start of the connection, as in SMTPS. It establishes an encrypted channel immediately on a dedicated port and provides strong end-to-end protection. Opportunistic TLS, so called because it only applies if an upgrade is possible, is enabled via STARTTLS protocol extension and attempts to convert the plaintext connection to TLS-protected one. Its effectiveness, however, can be undermined by misconfigured clients or servers.

B. STARTTLS Mechanism

STARTTLS is an extension for email protocols that allows a client and a server to upgrade an existing plaintext connection to a TLS-protected session. It is supported in SMTP, IMAP, and POP3, allowing encryption to be established over ports that were originally designated for plaintext communication. The STARTTLS process begins with the client connecting to the server over the standard protocol port (e.g., 587 for SMTP, 143 for IMAP, 110 for POP3). Once a client connects to a server over the standard protocol port, the server advertises its capabilities, including the possibility of security upgrade via STARTTLS, then the client attempts to upgrade the plaintext connection to TLS using the STARTTLS command. After that the server acknowledges the request and initiates the TLS handshake to establish a secure channel. Upon successful completion of the handshake, all subsequent communication is encrypted.

A simplified overview of this workflow is illustrated in Figure 1, showing the chronological exchange between client and server: the client initiates the connection, the server advertises STARTTLS capability, the client initiates the STARTTLS process, which triggers the TLS handshake, after which the session continues securely.

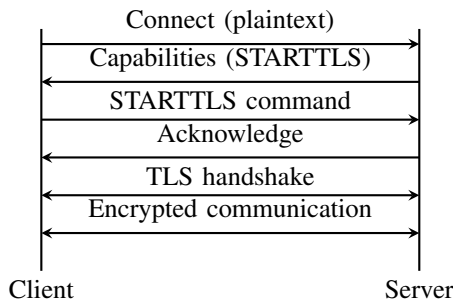


Fig. 1. STARTTLS handshake overview between client and server.

C. Vulnerability of Opportunistic TLS

To define the threat model, we consider an active network attacker, commonly referred to as a man-in-the-middle, who can intercept, modify, or drop TCP packets between the email client and server. Such an attacker may be present on public

Wi-Fi networks or within an Internet Service Provider (ISP), but is assumed not to be able to break cryptographic primitives or forge valid certificates.

While opportunistic TLS via STARTTLS enables upgrading existing plaintext connections to encrypted sessions, its security relies on proper enforcement by both clients and servers. Under the assumptions of our threat model, this reliance creates clear vulnerabilities, since an attacker with network access can interfere with the negotiation process. A notable example is the STARTTLS downgrade attack [2], in which an attacker removes the STARTTLS command from the server’s advertised capabilities, forcing the client to continue in plaintext. Such attacks exploit the opportunistic nature of the protocol, where encryption is not mandatory and connections may fall back to unencrypted communication if the upgrade cannot be completed.

The impact of these vulnerabilities depends on the behavior of the client and server. Clients that strictly require TLS are not susceptible to downgrade attacks. Similarly, misconfigured servers that fail to correctly enforce TLS increase the risk of compromise.

Common misconfigurations in the email ecosystem include [3]:

- Servers and clients supporting no-TLS authentication as an option.
- Clients automatically falling back to plaintext when TLS negotiation fails.
- Client’s “auto-detect” feature may provide an insecure set of configuration parameters.

These factors together illustrate why opportunistic TLS, despite its widespread adoption, can leave email transmissions exposed to interception and manipulation.

D. Related Work

Email security has been the subject of extensive research, with numerous studies investigating both protocol-level and deployment-level vulnerabilities.

Early work [8] measured the use of STARTTLS across SMTP servers, providing initial insights into the TLS deployment via security upgrade in practice.

Mayer et al. [9] evaluated the security of email protocols by analyzing the underlying TLS primitives. Through active scanning of all email ports across the IPv4 address space, they observed that many servers still accept weak export-grade ciphers, and a majority of certificates are self-signed. In contrast, our work focuses on how TLS can be completely circumvented, rather than on weaknesses in the TLS protocol primitives.

A recent large-scale study on email confidentiality and integrity [10] shows that STARTTLS support has become common among mail servers, however issues with certificate validity persist, highlighting ongoing gaps between theoretical security guarantees and real-world deployment. While the widespread adoption of STARTTLS represents a clear improvement compared to scenarios where no TLS is used,

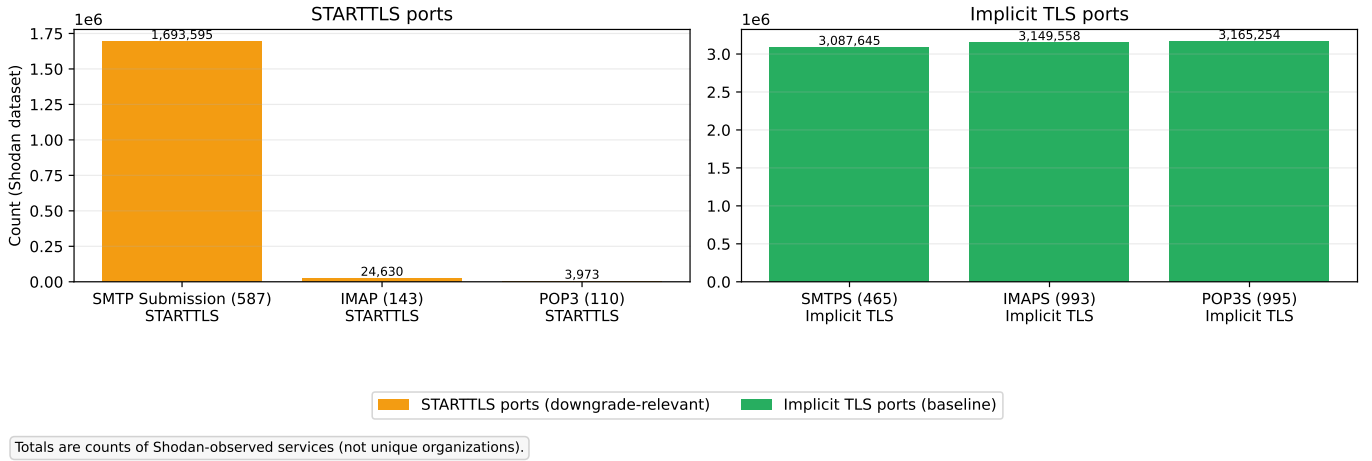


Fig. 2. Distribution of email services observed via Shodan across STARTTLS and implicit TLS ports.

it remains insufficient in the presence of an active network attacker, as discussed in this work.

In [11] authors propose a tool for pentesters to evaluate email servers and clients in relation to TLS security, including checking for STARTTLS stripping attacks. However, a key drawback is the tool’s complicated technical setup, which limits its adoption by non-experts.

In [3] the authors present a comprehensive study of the email ecosystem, analyzing client-side implementations, IT administrators’ setup guides, and server-side deployments. They identify several critical security flaws that can lead to credential theft, including improper handling of security downgrades and certificate validation by some email clients. The study also highlights risks introduced by auto-detect features and UI designs that can inadvertently compromise security. Furthermore, many setup guides either fail to provide clear instructions for securing configurations or recommend insecure practices such as accepting all certificates. Building on these insights, our work focuses on practical tools to improve transparency and security. In this way, we complement prior research by moving from measurement and analysis to actionable solutions for end users and administrators. To contextualize our approach, existing TLS-related assessment tools, all of which differ from our proposed solution, are discussed in detail in Section V, where they are reviewed in relation to the design and functionality of our service.

III. PASSIVE MEASUREMENT OF EMAIL SERVERS SECURITY

This section presents our passive measurement methodology for identifying Internet-facing email services with insecure configurations that may be potentially vulnerable to downgrade attacks. Using data collected by Shodan, we focus on observable server-side configurations related to opportunistic TLS, including the advertisement of encryption capabilities and

authentication mechanisms. The motivation for this analysis is to assess how prevalent STARTTLS-related misconfigurations are in practice. We aim to estimate the exposure of email servers to downgrade attacks and potential plaintext credential transmission.

A key concern on the server side is the continued support for insecure configurations that allow plaintext authentication. To investigate this issue, we consider the following research question: *How many email servers accept plaintext authentication?*

To address this question, we adopt a passive measurement methodology using Shodan [12], a search platform for Internet-connected devices. Instead of directly interacting with live email services, we query the Shodan API to analyze data already collected in its database. We apply this approach for SMTP, IMAP, and POP3, in order to obtain a broader view of authentication and encryption practices in real-world deployments. To illustrate our methodology, we focus here on SMTP submission services on port 587 as a representative example. For each protocol under study, we construct protocol- and product-specific queries. In the SMTP case, for a given product p , we first retrieve the total number of exposed services using the query `port:587 product:"p"`. We then identify servers advertising authentication mechanisms by extending this query to `port:587 product:"p" ("250-AUTH" OR "AUTH") (PLAIN OR LOGIN)`. This approach allows us to estimate the proportion of servers that may offer authentication prior to TLS negotiation.

Our queries capture advertised capabilities as observed in real-world client-server interactions during protocol negotiation. In the SMTP protocol, a client first issues an EHLO command to request server capabilities, to which the server responds with a list of supported extensions. The presence of plaintext authentication mechanisms in this response may indicate that credentials could be transmitted without encryption.

However, this observation alone does not confirm an exploitable vulnerability. In practice, some servers enforce TLS by rejecting authentication attempts prior to a successful STARTTLS negotiation, by responding with an error such as:

```
530 5.7.0 Must issue a STARTTLS command first
```

As a result, our measurements identify potentially vulnerable configurations rather than definitively insecure deployments. Our Shodan search results indicate that a non-negligible fraction of email servers may allow plaintext credential transmission under certain conditions (see Fig. 2). The results show that STARTTLS is widely deployed, particularly on the SMTP submission port (587), where a large number of servers advertise upgrade capabilities. In contrast, implicit TLS services on dedicated ports such as 465, 993, and 995 appear in significantly higher numbers overall, reflecting their role as a current baseline for secure-by-default communication. At the same time, the continued prevalence of STARTTLS-enabled services highlights the relevance of downgrade attacks, as these deployments rely on successful negotiation rather than enforcing encryption from the outset.

IV. SERVER ADMINISTRATOR CONFIGURATION SCANNER

We developed a lightweight Bash script that helps email server administrators verify secure server-side TLS configurations, ensuring resilience against STARTTLS downgrade attacks. The goal of the tool is to detect misconfigurations in popular mail servers and provide actionable guidance to enforce mandatory TLS for authentication and message transmission.

A. Supported Servers and Detection Logic

The scanner currently supports Postfix and Dovecot, two widely used SMTP and IMAP/POP3 servers on Linux. Upon execution, the script first determines which mail server software is installed by checking for the presence of the `postconf` and `doveconf` utilities. This allows the tool to run safely on any Linux system without prior knowledge of the installed mail stack; servers that are not present are simply skipped.

For Postfix, the scanner reads the active configuration via `postconf` and inspects `smtpd_tls_security_level` and `smtpd_tls_auth_only` in `main.cf`. Critically, it also parses `master.cf` to detect per-service overrides. Email submission services on port 587 frequently override global settings with service-specific parameters (e.g., `-o smtpd_tls_security_level=may`), which can silently re-introduce vulnerabilities even when the global configuration is secure. The scanner explicitly checks for such overrides and reports them separately.

For Dovecot, the scanner reads the active configuration via `doveconf` and checks `disable_plaintext_auth`, `ssl`, and `auth_mechanisms`. It handles both monolithic configurations (`dovecot.conf`) and the common split-file layout where settings are distributed across `conf.d/10-auth.conf`, `conf.d/10-ssl.conf`, and

TABLE I
CONFIGURATION PARAMETERS EVALUATED BY THE SCANNER

Parameter	Warning	Recommended
<i>Postfix</i>		
<code>smtpd_tls_security_level</code>	may	encrypt
<code>smtpd_tls_auth_only</code>	no	yes
<i>Dovecot</i>		
<code>disable_plaintext_auth</code>	no	yes
<code>ssl</code>	optional	required
<code>auth_mechanisms</code>	PLAIN / LOGIN	disable or enforce TLS

related files. Each detected issue is reported with the exact file path where the insecure setting was found, enabling administrators to locate and fix the problem directly.

The complete set of evaluated parameters and the corresponding warning conditions are summarized in Table I.

B. Usage and Output

Administrators can execute the script in console mode or generate a machine-readable JSON report by specifying an output file with the `-o` flag. The JSON format enables integration into automated compliance pipelines or monitoring dashboards.

An example output highlighting an insecure configuration in Dovecot and the recommended fix is shown below:

```
[Dovecot] disable_plaintext_auth = no
Reason: Allows cleartext authentication
Recommendation:
  disable_plaintext_auth = yes
File: /etc/dovecot/conf.d/10-auth.conf
```

Each detected issue includes:

- The configuration option found;
- The reason why it may introduce a security risk;
- A recommended fix to enforce secure TLS usage;
- The file location of the configuration

This output allows administrators to quickly identify and remediate misconfigurations that could otherwise enable STARTTLS downgrade attacks, ensuring that authentication and message transport remain encrypted.

V. SELFTEST SERVICE FOR EMAIL CLIENTS

While previous research has highlighted the vulnerabilities of opportunistic TLS in email clients, little attention has been given to empowering end users with tools to independently assess their exposure. Most existing solutions either test server-side configurations or require complex local setups, making them inaccessible to the average user or non-security-expert administrators. To bridge this gap, we designed and implemented a public, client-facing selftest service.

A. Comparison with Existing Tools

The landscape of email security testing tools is fragmented. As shown in Table II, existing tools solve different parts of the equation but leave a gap for end-user client testing.

- **EAST [11]:** A comprehensive toolkit for researchers that simulates MITM attacks on both clients and servers.

TABLE II
COMPARISON OF CLIENT-SIDE SECURITY TESTING TOOLS

Feature	EAST	CheckTLS	BadSSL	Ours
Target	Client	Server	Browser	Client
Interface	CLI / VM	Web form	Website	Web app
Audience	Research	Admins	Users	All
Strip test	Yes	No	N/A	Yes
Setup	Local lab	None	None	None
Protocols	All Mail	SMTP	HTTPS	Mail
Limitation	Proxy	MTA only	Web only	IP link

However, it requires a local lab environment, proxy setup, and custom certificate authorities, making it unsuited for end users.

- **CheckTLS [13]:** A widely used web service that checks server-to-server (MTA) encryption. It verifies if a domain accepts TLS but cannot test how an end user’s MUA behaves.
- **BadSSL [14]:** The gold standard for testing web browser reactions to broken TLS (e.g., expired or self-signed certificates). While excellent for HTTP, it does not support email protocols.

Our Selftest Service fills this gap by acting as a “BadSSL for email clients,” providing a web-based, zero-setup testing environment for SMTP and IMAP.

B. Design Goals

The development of the selftest service was driven by three primary design goals:

- 1) **End-user usability:** The tool must not require the installation of local proxies (e.g., mitmproxy [15]) or custom root certificates, which are significant hurdles for non-technical users.
- 2) **Realistic attack simulation:** The service must accurately mimic the behavior of an active network attacker directly from the server side, reliably triggering potential downgrade vulnerabilities in the client.
- 3) **Minimal setup effort:** Users should only need to configure a standard email account in their client using provided test credentials to perform the evaluation.

C. Methodology: Server-Side Simulation

The fundamental premise of our service is to shift the attack simulation from an intermediate network proxy to the destination mail server itself. We implemented a custom multi-protocol mail server in Python that deliberately manipulates the protocol exchange based on the test case selected by the user in the WebUI.

To force clients into opportunistic TLS negotiation, the service employs *implicit TLS blocking*. When an email client attempts to connect to dedicated implicit TLS ports (e.g., 993 for IMAPS, 465 for SMTPS), the server immediately terminates the TCP connection. This rejection forces modern clients to fall back to STARTTLS ports (143 for IMAP, 587 for SMTP), where the downgrade behavior can be observed and manipulated by our server logic.

1) *The Pre-Authentication Session Binding Problem:* A significant technical challenge of server-side simulation is session tracking. Unlike HTTP, which utilizes cookies or headers to maintain session state from the very first request, SMTP and IMAP lack built-in session identifiers prior to authentication. When a client connects, the server only knows its IP address. However, the decision to strip STARTTLS (e.g., in scenario T1) must happen during the initial EHLO or CAPABILITY phase, long before the client sends its username.

We solve this *pre-authentication session binding problem* using a two-phase approach:

- **Phase 1: Pre-authentication (IP-based):** When a user starts a test session via the WebUI, the system creates a Time-To-Live (TTL) override mapping the user’s public IP address to the selected test mode (e.g., T1). When an incoming connection arrives on the mail server, it reads this mapping to determine how to manipulate the capabilities response. This phase is susceptible to issues with NAT, CGNAT (mobile carriers), or dynamic IP changes, representing the primary limitation of our approach.
- **Phase 2: Post-authentication (Username-based):** Once the client proceeds to send the AUTH or LOGIN command, the server extracts a unique, cryptographically secure session ID embedded directly within the username (e.g., test-SESSION@domain). From this point forward, the event is securely bound to the user’s browser session, guaranteeing accurate event correlation even if the IP address changes or is shared among multiple users.

A potential technical solution to completely eliminate the reliance on IP-based tracking is the implementation of per-session subdomains (e.g., session-123.selftest.example.com) coupled with a wildcard TLS certificate (*.selftest.example.com). By instructing the client to connect to a unique hostname, the server could theoretically identify the session without relying on the source IP. For implicit TLS connections, the session ID could be extracted immediately from the Server Name Indication (SNI) during the TLS handshake. However, because STARTTLS connections begin in plaintext and SMTP/IMAP protocols lack an equivalent to HTTP’s Host header, the server cannot see the requested hostname before the EHLO phase. To make this work for STARTTLS, the wildcard DNS would need to be backed by a massive pool of IP addresses (e.g., an IPv6 /64 subnet), allowing the server to bind the session based on the unique *destination* IP address rather than the client’s source IP.

2) *Protocol Scope:* While the original paper [3] evaluates SMTP, IMAP, and POP3, our Selftest Service currently focuses exclusively on SMTP and IMAP. POP3 is increasingly considered a legacy protocol. Including it would require users to configure an additional set of ports and behaviors, increasing the configuration effort and likely reducing the completion rate for a public self-test. SMTP submission and IMAP cover the vast majority of modern, real-world email client configurations.

3) *Bypassing Client Heuristics and ISPDB*: A common challenge when testing email clients is their reliance on internal databases, such as Mozilla’s ISPDB [16], to automatically configure known mail providers. If a client finds a match in its database, it may use pre-defined secure settings and bypass the network-based auto-detect process entirely. To guarantee that the client evaluates the live network conditions, our service utilizes dynamic IP-based domains (e.g., `<ip>.nip.io`). The `nip.io` service [17] acts as a wildcard DNS resolver that maps any IP address embedded in the hostname directly back to that IP. Because these auto-generated domains are virtually infinite and unique to the testing instance’s IP address, they will never be listed in static ISP databases. By presenting this unknown domain, we effectively circumvent local caches and force the client into a true heuristic auto-detect fallback.

D. Architecture and System Overview

The system architecture consists of two main components: a web-based User Interface (WebUI) built with FastAPI [18], and a custom Python-based multi-protocol mail server handling SMTP and IMAP traffic.

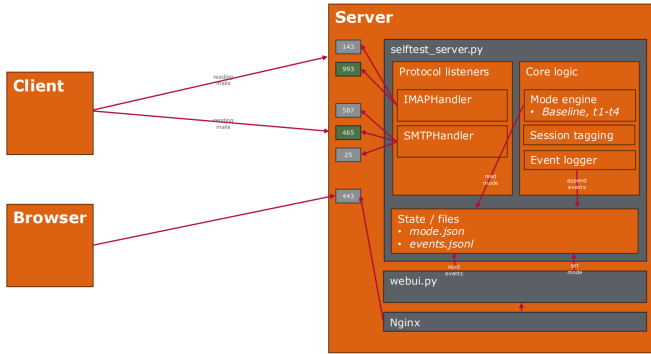


Fig. 3. Architecture of the Selftest Service

To manage state efficiently and reliably between these asynchronous components, the system utilizes a dual-storage architecture rather than a complex relational database:

- **Event Log (JSONL)**: Protocol events (e.g., `connect`, `ehlo`, `auth_command`) are recorded in an append-only JSON Lines format, guaranteeing crash safety. Furthermore, the system is built with strict *privacy-by-design* principles. Because it operates as a public service, it is critical that user credentials remain protected. Passwords are discarded immediately at the protocol parser level and are never written to memory or disk. The logs strictly record only the session ID, event type, and a boolean `tls` flag.
- **Mode Store (JSON)**: Mutable state, such as active IP overrides and guided test run progress, is maintained in a monolithic JSON file using atomic write operations (writing to a temporary file and renaming) to prevent concurrent corruption.

E. Security Properties Tested

The service evaluates clients against four distinct disruption scenarios, originally defined by Poddebniak et al. [3]:

- **T1 (STARTTLS Stripping)**: The server suppresses the STARTTLS capability in its initial greeting, simulating a classic stripping attack.
- **T2 (Handshake Drop)**: The server advertises STARTTLS but drops the TCP connection immediately after the client sends the STARTTLS command.
- **T3 (Handshake Failure)**: The server simulates a cryptographic failure during the TLS handshake (e.g., presenting an invalid certificate).
- **T4 (Post-Handshake Manipulation)**: The server injects invalid data or drops the connection after a successful TLS handshake, testing if the client attempts a plaintext fallback.

Based on the client’s reaction, the Verdict Engine computes a deterministic outcome by analyzing specific signals within the event log:

- **auth_plain count**: If the server observes an authentication attempt over an unencrypted channel (plaintext credentials exposure), the verdict is **FAIL**.
- **auth_tls count**: If authentication only occurs over an active TLS session, the verdict is **PASS**.
- **retry_like signal**: If the server observes 6 or more connection attempts without any authentication reaching the server, the client is assumed to be stuck in a retry loop. The verdict is **INCONCLUSIVE** (the client behaved securely by failing closed, but the service becomes unusable).
- **starttls_refused_like signal**: Identifies if the server successfully disrupted the TLS upgrade as dictated by the testcase behavior. If the client aborts the connection upon seeing this, the verdict is also **INCONCLUSIVE**.
- **NOT_APPLICABLE**: The user reported via the WebUI that the client could not connect at all (e.g., due to strict local firewall rules or hardcoded implicit TLS requirements), meaning the specific downgrade step could not be measured.
- **SKIPPED**: The step was manually skipped by the user in Guided Mode.

F. Example Results and Interpretation

Users interact with the service primarily through a “Guided Mode,” which sequentially executes a 9-step evaluation. The sequence is structured around two disruption semantics:

- **Immediate disruption**: The selected test mode (e.g., T1) is active from the very first connection attempt. This tests how the client reacts when STARTTLS is unavailable from the outset, for example during initial account setup or when connecting to an unfamiliar server.
- **Two-phase disruption**: The service first allows a successful TLS-protected authentication (baseline), then activates the disruption mode for subsequent connections.

This simulates an attacker who begins interfering with an already-established account, testing whether the client silently downgrades on reconnection after having previously succeeded with TLS.

The 9-step guided sequence begins with a secure baseline (Step 1) to verify that the client can connect and authenticate over TLS under normal conditions. Steps 2–5 apply immediate disruptions for T1 through T4, respectively. Steps 6–9 repeat T1 through T4 using two-phase semantics, where each step first confirms a successful TLS connection before activating the disruption. Each step uses fresh, unique session credentials to prevent cross-contamination between test modes.

For researchers and administrators who wish to isolate specific behaviors, an “Advanced Mode” is also available, allowing for the manual selection of individual testcases and scenarios without executing the full suite.

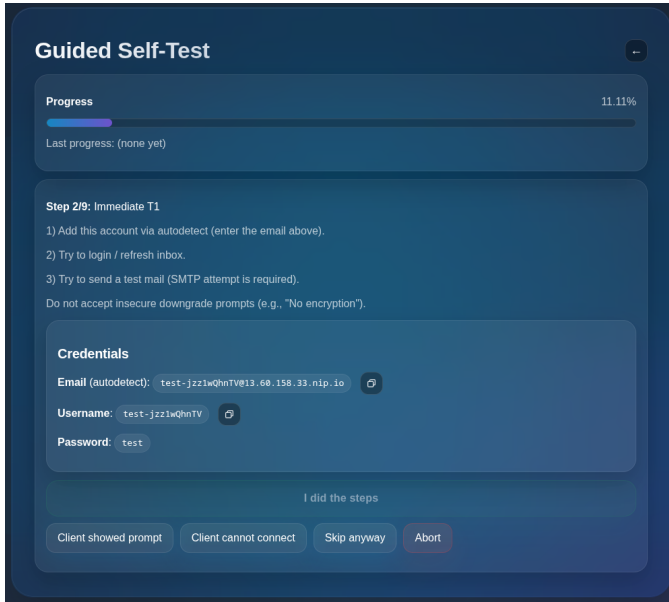


Fig. 4. The Guided Mode interface of the Selftest Service

For example, when evaluating a vulnerable client against the T1 scenario, the user configures the client with the provided session credentials and triggers a mail synchronization. The WebUI polls the event log and displays the raw protocol sequence to the user in real-time: `connect` → `ehlo` (where STARTTLS is intentionally stripped) → `auth_command` (executed without TLS). Because the authentication occurred in plaintext, the WebUI immediately presents a FAIL verdict.

Furthermore, passive server logs alone cannot determine whether a client displayed a warning to the user before downgrading. To address this, the WebUI allows users to manually report if their client displayed a security warning during the process. This adjusts the final verdict to WARN, adding crucial context about the client’s UI behavior that the server side cannot natively observe.

G. Limitations and Deployment Constraints

While the server-side simulation approach offers significant usability benefits, it introduces specific technical and operational limitations:

- **Destination Server, Not a Proxy:** Unlike local MITM proxies (such as EAST) that intercept traffic between the user and their actual email provider (e.g., Gmail, Outlook), our service acts as the destination server itself. Therefore, it tests how a client behaves *in general* when a server acts maliciously, but it cannot verify if the client uses hardcoded security pinning for specific major providers. While it would be technically possible to implement a proxy-based approach to test connections to real providers, this would require users to install custom Root Certificate Authorities (CAs) on their devices to bypass certificate validation during the MITM interception. This directly contradicts our primary design goal of zero-setup, end-user usability.
- **Operational Constraints (Port Binding):** To manipulate protocol handshakes accurately, the custom Python mail server must bind directly to the privileged email ports (25, 143, 465, 587, 993). This creates a strict deployment constraint: the service cannot be hosted on a machine or IP address that already runs a standard mail transfer agent (MTA) like Postfix or Dovecot, as the ports would conflict. It requires a dedicated IP address or virtual machine for deployment.

VI. PRELIMINARY EMAIL CLIENT EVALUATION

To validate the practical utility of both our selftest service and the traditional MITM-based methodology, we conducted an initial evaluation of a widely used email client. While a comprehensive multi-client study is planned as future work, these preliminary results demonstrate the effectiveness of our tools and provide first insights into the current state of email client security.

A. Evaluation Setup

We evaluated Mozilla Thunderbird 140.4.0esr on Kali GNU/Linux Rolling (x86_64) using the mitmproxy-based methodology (transparent proxy with iptables redirection) against our vulnerable test server (mail.nsipmail.de). The server was deliberately configured to accept plaintext authentication on STARTTLS ports (587 for SMTP, 143 for IMAP, 110 for POP3), while also providing secure implicit TLS endpoints (465, 993, 995) with valid Let’s Encrypt certificates. This dual configuration ensures that clients have the opportunity to choose either secure or insecure paths, mirroring a realistic deployment where both options coexist.

For each protocol, Thunderbird was configured with manual STARTTLS settings pointing to the respective STARTTLS port. The test cases T1–T4, implemented as mitmproxy scripts from the original research codebase [3], were executed sequentially. Traffic was captured to disk and analyzed post-hoc for any occurrence of plaintext credentials (AUTH PLAIN, LOGIN, or USER/PASS commands transmitted without TLS).

TABLE III
THUNDERBIRD 140.4.0ESR EVALUATION RESULTS (MANUAL CONFIG)

Protocol	Port	T1	T2	T3	T4
IMAP	143	✓	✓	✓	△
SMTP	587	△	✓	✓	✓
POP3	110	✓	✓	✓	✓
✓ Secure △ User-dependent × Vulnerable					

B. Results

The results are summarized in Table III. Thunderbird demonstrated strong resilience across the majority of tested scenarios.

1) *Secure Behavior (10 of 12 Scenarios)*: In ten out of twelve test combinations, Thunderbird refused to transmit credentials in plaintext. Under T1 (STARTTLS stripping), the client detected the missing capability and aborted the connection for both IMAP and POP3. Under T2 (ServerHello replacement) and T3 (STARTTLS command rejection), the client consistently failed closed across all three protocols, with connections either timing out or producing error messages without any authentication attempt over the unencrypted channel. Examination of the captured traffic confirmed the absence of AUTH, LOGIN, or USER/PASS commands in plaintext.

2) *User-Dependent Behavior (2 of 12 Scenarios)*: Two scenarios exhibited behavior where the security outcome depends on user interaction:

SMTP T1 (STARTTLS Stripping): Thunderbird correctly aborted the SMTP session when STARTTLS was stripped from the EHLO response. However, the error dialog presented to the user explicitly suggested: “Switch off STARTTLS for that server.” If a user follows this recommendation, subsequent connections would proceed entirely without encryption. While the client itself does not automatically downgrade, the suggested remediation effectively instructs the user to disable the very protection being tested.

IMAP T4 (Post-Handshake Manipulation): When mitm-proxy intercepted the TLS session and presented its own untrusted certificate, Thunderbird displayed a certificate warning dialog with the option to “Confirm Security Exception.” Clicking this button would allow the connection to proceed through the MITM, potentially exposing credentials. Notably, no credentials were transmitted unless the user actively bypassed the warning.

C. Comparison with Prior Work

Our findings are broadly consistent with the results reported by [3], where Thunderbird was classified as secure under manual configuration for the STARTTLS downgrade scenarios. The original study found that only 8 out of 49 tested clients offered “opportunistic TLS” (attempting TLS but continuing without it upon failure) in manual configuration mode. Our results confirm that Thunderbird does not exhibit this behavior.

However, the user-dependent scenarios we identified highlight a dimension not fully captured by binary pass/fail classifications. The SMTP T1 error message effectively functions as a social engineering vector: a confused user following the

client’s own suggestion could inadvertently disable STARTTLS entirely. This observation aligns with findings in [3] regarding UI designs that “can inadvertently compromise security,” and suggests that client security evaluations should consider not only protocol-level behavior but also the guidance provided to users during failure states.

The auto-detect evaluation, where the original study found 14 out of 30 clients silently downgrading, remains untested in our preliminary work and is a priority for future evaluation using the selftest service.

VII. DISCUSSION AND FUTURE WORK

In this section, we reflect on the limitations of our approach, revisit the research questions that motivated this work in light of our findings from Section VI, and outline directions for future research.

A. Limitations

1) *Server-Side Simulation vs. True MITM*: Our selftest service deliberately simulates attacker behavior from the destination server rather than deploying a true man-in-the-middle proxy. While this design achieves our primary goal of zero-setup usability, it introduces a fundamental trade-off in fidelity. In particular, the T2 scenario (replacing the TLS ServerHello with cleartext after the client sends a ClientHello) cannot be faithfully reproduced without on-path interception. Our server-side approximation (accepting the STARTTLS command and then failing the TLS handshake) triggers the same client decision point in most cases but may not exercise identical code paths in all implementations. Similarly, because the service acts as the destination server itself, it cannot verify whether a client applies provider-specific security pinning (e.g., hardcoded TLS requirements for Gmail or Outlook).

2) *IP-Based Session Binding*: The pre-authentication phase of our session binding relies on the client’s source IP address. This introduces complications when users connect from behind Network Address Translation (NAT), Carrier-Grade NAT (CG-NAT), or shared institutional networks, where multiple users may share the same public IP. Although the post-authentication phase resolves this ambiguity via a unique session identifier embedded in the username, the initial protocol manipulation (e.g., stripping STARTTLS in T1) must occur before any credentials are sent. In practice, this means that concurrent users behind the same NAT gateway may interfere with each other’s test sessions.

3) *ISPDB and Client Discovery Mechanisms*: Some email clients, most notably Mozilla Thunderbird, query external provider databases such as the ISPDB [16] during account setup. If the client finds a matching configuration entry, it may bypass the network-based auto-detect process entirely and use pre-defined secure settings. Because ISPDB queries are outbound requests to third-party hosts, our service cannot intercept or influence them. While our use of dynamically generated nip.io domains mitigates this issue for most clients, it does not guarantee that all clients will enter the

heuristic auto-detect path that our service is designed to evaluate.

4) *Passive Measurement Scope*: Our Shodan-based server measurement identifies *potentially* vulnerable configurations rather than confirmed exploitable deployments. Servers that advertise plaintext authentication mechanisms may still reject authentication attempts prior to STARTTLS negotiation (e.g., by responding with 530 Must issue a STARTTLS command first). Without active probing, our methodology cannot distinguish between servers that merely advertise insecure capabilities and those that genuinely accept plaintext credentials. Nevertheless, the prevalence of such configurations highlights the relevance of STARTTLS-related vulnerabilities.

B. Research Questions Revisited

Our preliminary evaluation of Thunderbird (Section VI) partially addresses the first research question, namely whether current email client versions remain vulnerable to STARTTLS downgrade attacks. For this client in manual configuration mode, the answer is largely negative: the client fails securely in 10 out of 12 scenarios. However, the user-dependent behaviors identified in the remaining two scenarios reveal a secondary attack surface that relies on social engineering rather than protocol-level exploitation, suggesting that binary pass/fail classifications may not fully capture the security posture of a client.

Several research questions remain open. The question of how different clients implement auto-detect and which patterns are most vulnerable could not be fully addressed within the scope of this work, as auto-detect testing requires either a controlled network environment or per-client workarounds for ISPDB interference. Similarly, a systematic evaluation of certificate validation weaknesses (e.g., acceptance of self-signed, expired, or domain-mismatched certificates) was planned but deprioritized in favor of the selftest service development. Finally, the impact of real-world setup guides on user security, a topic explored in the original study [3], represents a complementary direction that our tools could support in future work by providing a standardized methodology for evaluating the configurations recommended by institutional IT departments.

C. Future Work

1) *Multi-Client Evaluation*: The most impactful next step is to conduct a systematic evaluation across a broad range of email clients and platforms using the selftest service. This includes popular desktop clients (e.g., Apple Mail, Outlook), mobile clients (e.g., K-9 Mail, Samsung Email), and web-based clients. A direct comparison with the findings of [3], which tested 49 clients, would reveal whether vendors have addressed the reported vulnerabilities in current versions.

2) *Auto-Detect Testing*: Extending the evaluation to cover auto-detect configurations is critical, as this is where the original study identified the most severe vulnerabilities: 14 out of 30 auto-detect-capable clients silently downgraded without user knowledge [3]. Achieving this in a public self-test context will likely require a hybrid approach combining server-side

simulation with controlled network environments for clients that rely on external discovery mechanisms.

3) *Certificate Validation Testing*: A dedicated test suite for certificate validation (e.g., self-signed, expired, and domain-mismatched certificates) would complement the current STARTTLS-focused evaluation. This could be integrated into the selftest service by extending the TLS configuration to present deliberately invalid certificates during specific test scenarios.

4) *POP3 Support*: While POP3 was deprioritized due to its declining usage and the additional configuration burden it imposes on users, adding POP3 support would improve completeness and enable direct comparisons with the full T1–T4 test matrix from the original study.

5) *Per-Session Subdomains*: As discussed in Section V, implementing per-session subdomains backed by wildcard DNS and a pool of IPv6 addresses could eliminate IP-based session binding entirely. This would improve reliability for users behind NAT/CGNAT and enable concurrent testing by multiple users without interference.

VIII. CONCLUSION

This work addressed the gap between the theoretical security guarantees of TLS in email and the practical challenges of verifying correct implementations. We presented three main contributions targeting both sides of the email security equation.

First, we developed a web-based selftest service that enables end users to evaluate the STARTTLS downgrade resilience of their email clients without requiring local proxy installation, custom certificate authorities, or specialized technical knowledge. By shifting the attack simulation from a network-level MITM proxy to the destination mail server itself, the service achieves zero-setup usability while still accurately reproducing the decision points tested by the T1–T4 disruption scenarios. The guided 9-step evaluation workflow, combined with a deterministic verdict engine, provides interpretable and comparable results across different clients.

Second, we provided a lightweight configuration scanner for email server administrators that audits Postfix and Dovecot deployments for common misconfigurations, including per-service overrides in submission configurations, that could enable plaintext credential transmission under STARTTLS downgrade conditions.

Third, we conducted a preliminary evaluation of Mozilla Thunderbird against the full T1–T4 test matrix across IMAP, SMTP, and POP3. Thunderbird proved largely resilient, failing securely in 10 out of 12 tested scenarios. However, two user-dependent behaviors were identified: an error message that explicitly suggests disabling STARTTLS (SMTP T1) and a certificate exception dialog that allows users to bypass TLS validation (IMAP T4). These findings highlight that client security evaluations should consider not only protocol-level behavior but also the guidance provided to users during failure states, as misleading suggestions can function as social engineering vectors.

Additionally, a passive measurement of Internet-facing email servers using Shodan revealed that a non-negligible fraction of deployments continue to advertise plaintext authentication mechanisms on STARTTLS ports, underscoring the continued real-world relevance of downgrade attacks.

Together, these tools, findings, and methodology provide a foundation for broader evaluations across diverse email clients and platforms, ultimately contributing to improved transparency and security in the email ecosystem.

ACKNOWLEDGMENT

The authors thank the "Network Security in Practice" seminar for valuable guidance and discussions that contributed to this work.

REFERENCES

- [1] L. Ceci, (2026, 01) Number of e-mail users worldwide 2026. [Online]. Available: <https://www.statista.com/statistics/255080/number-of-e-mail-users-worldwide/>
- [2] V. Dukhovni, "Opportunistic Security: Some Protection Most of the Time," RFC 7435, Dec. 2014. [Online]. Available: <https://www.rfc-editor.org/info/rfc7435>
- [3] K. Tang, C. Tu, S. Mak, and S. Chau, "A multifaceted study on the use of tls and auto-detect in email ecosystems," 01 2025.
- [4] D. J. C. Klensin, "Simple Mail Transfer Protocol," RFC 5321, Oct. 2008. [Online]. Available: <https://www.rfc-editor.org/info/rfc5321>
- [5] D. M. T. Rose and J. G. Myers, "Post Office Protocol - Version 3," RFC 1939, May 1996. [Online]. Available: <https://www.rfc-editor.org/info/rfc1939>
- [6] A. Melnikov and B. Leiba, "Internet Message Access Protocol (IMAP) - Version 4rev2," RFC 9051, Aug. 2021. [Online]. Available: <https://www.rfc-editor.org/info/rfc9051>
- [7] G. W. Developers, "Imap, pop, and smtp," 2026, accessed: March 27, 2026. [Online]. Available: <https://developers.google.com/workspace/gmail/imap/imap-smtp>
- [8] Z. Durumeric, D. Adrian, A. Mirian, J. Kasten, E. Bursztein, N. Lidzborski, K. Thomas, V. Eranti, M. Bailey, and J. A. Halderman, "Neither snow nor rain nor mitm...: An empirical analysis of email delivery security," in *Proceedings of the 2015 Internet Measurement Conference*, ser. IMC '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 27–39. [Online]. Available: <https://doi.org/10.1145/2815675.2815695>
- [9] W. Mayer, A. Zauner, M. Schmiedecker, and M. Huber, "No need for black chambers: Testing tls in the e-mail ecosystem at large," in *2016 11th International Conference on Availability, Reliability and Security (ARES)*, 2016, pp. 10–20.
- [10] B. Blechschmidt and B. Stock, "Extended hell(o): A comprehensive Large-Scale study on email confidentiality and integrity mechanisms in the wild," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 4895–4912. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/blechschmidt>
- [11] D. Poddebniak, F. Ising, H. Böck, and S. Schinzel, "Why TLS is better without STARTTLS: A security analysis of STARTTLS in the email context," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 4365–4382. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/poddebniak>
- [12] Shodan, "Shodan: The search engine for internet-connected devices," <https://www.shodan.io/>, accessed: 2026-03-28.
- [13] CheckTLS, "CheckTLS – email security testing," <https://www.checktls.com/>, accessed: 2026-03-28.
- [14] A. King and L. Garron, "badssl.com – testing clients against bad SSL configurations," <https://badssl.com/>, accessed: 2026-03-28.
- [15] mitmproxy project, "mitmproxy – an interactive HTTPS proxy," <https://mitmproxy.org/>, accessed: 2026-03-28.
- [16] Mozilla, "Thunderbird autoconfiguration – ISPDB," <https://wiki.mozilla.org/Thunderbird:Autoconfiguration>, accessed: 2026-03-28.
- [17] nip.io, "nip.io – dead simple wildcard DNS for any IP address," <https://nip.io/>, accessed: 2026-03-28.
- [18] S. Ramírez, "FastAPI – modern, fast web framework for building APIs with Python," <https://fastapi.tiangolo.com/>, accessed: 2026-03-28.